

using `std::optional` and `std::move` to model cycle-approximate pipeline stalls provides a highly readable and elegant framework for cycle-approximate architectural modeling

In a cycle-approximate pipeline loop (typically driven by an `SC_METHOD` or `SC_THREAD` clocked sensitive block), a stage might contain a valid hardware instruction/packet, or it might be stalled (bubbled). Instead of tracking validity via manual, error-prone companion booleans (`bool stage1_valid`), wrapping the processing transaction inside a `std::optional` encapsulates both the state of data validity and the data itself into a single atomic variable.

Here is a complete SystemC implementation of a 3-stage instruction pipeline (Fetch, Decode, Execute) utilizing `std::optional` to manage pipeline bubbles and structural stalls dynamically.

1. Structural Pipeline Components

We will define a basic `Instruction` struct and create a pipeline module that updates its internal stages every clock cycle.

```
#include <systemc.h>
#include <optional>
#include <queue>
#include <iostream>
#include <iomanip>

// 1. Definition of the processing element
struct Instruction {
    uint32_t pc;
    std::string op;
    uint32_t delay_cycles; // Simulates structural stall dependency (e.g., multiplier latency)
};

SC_MODULE(PipelineModel) {
    sc_in<bool> clk;

    // Pipeline stages modeled via std::optional.
    // std::nullopt explicitly represents a pipeline "bubble" or "stall".
```

```

std::optional<Instruction> fetch_stage = std::nullopt;
std::optional<Instruction> decode_stage = std::nullopt;
std::optional<Instruction> execute_stage = std::nullopt;

// Architectural input stimulus queue
std::queue<Instruction> instruction_mem;

SC_CTOR(PipelineModel) {
    SC_METHOD(tick);
    sensitive << clk.pos();
    dont_initialize();

    // Seed mock instructions into our memory stream
    instruction_mem.push({0x1000, "ADD", 0});
    instruction_mem.push({0x1004, "MUL", 2}); // This instruction forces a 2-cycle stall!
    instruction_mem.push({0x1008, "SUB", 0});
    instruction_mem.push({0x100C, "AND", 0});
}

void tick() {
    std::cout << "\n--- [Cycle " << sc_time_stamp() << "]" << "\n";

    //-----
    // STAGE 3: EXECUTE
    //-----
    // .has_value() evaluates whether the pipeline stage contains an instruction
    if (execute_stage.has_value()) {
        if (execute_stage->delay_cycles > 0) {
            // Structural Stall Condition: The functional unit is busy!
            std::cout << "[EXECUTE] STALL: " << execute_stage->op
                << " busy. Remaining: " << execute_stage->delay_cycles << " cycles.\n";

            // Decrement the remaining busy execution cycles
            execute_stage->delay_cycles--;
        }
    }
}

```

```

        // Block previous stages from advancing by returning immediately
        print_pipeline_state();
        return;
    } else {
        // Clean Execution completion
        std::cout << "[EXECUTE] Retired Instruction: " << execute_stage->op
                  << " (PC: 0x" << std::hex << execute_stage->pc << std::dec << ")\n";

        // Clear the stage entirely, converting it back to std::nullopt (bubble)
        execute_stage = std::nullopt;
    }
} else {
    std::cout << "[EXECUTE] Bubble (Empty)\n";
}

//-----
// STAGE 2: DECODE
//-----
if (decode_stage.has_value()) {
    std::cout << "[DECODE] Processing: " << decode_stage->op << "\n";

    // Move structural ownership downstream into the execution phase
    execute_stage = std::move(decode_stage);

    // Explicitly clear the source stage to turn it back into a bubble
    decode_stage = std::nullopt;
} else {
    std::cout << "[DECODE] Bubble (Empty)\n";
}

//-----
// STAGE 1: FETCH
//-----
if (fetch_stage.has_value()) {
    std::cout << "[FETCH] Passing to Decode: " << fetch_stage->op << "\n";

```

```

        decode_stage = std::move(fetch_stage);
        fetch_stage = std::nullopt;
    }

    // Pull a fresh instruction from memory if available
    if (!instruction_mem.empty()) {
        fetch_stage = instruction_mem.front();
        instruction_mem.pop();
        std::cout << "[FETCH]   Fetched New: " << fetch_stage->op << "\n";
    } else {
        fetch_stage = std::nullopt;
    }

    print_pipeline_state();
}

// Helper debugging matrix output
void print_pipeline_state() {
    auto format_stage = [](const std::optional<Instruction>& stage) {
        return stage.has_value() ? stage->op : "---";
    };
    std::cout << "   [Pipeline Status] F: " << format_stage(fetch_stage)
        << " | D: " << format_stage(decode_stage)
        << " | E: " << format_stage(execute_stage) << "\n";
}
};

```

2. Main Simulation Harness

Binding a mock clock loop to execute the pipeline architecture trace:

```

int sc_main(int argc, char* argv[]) {
    // Generate an arbitrary 10ns simulation clock line
    sc_clock clk("clk", 10, SC_NS);
}

```

```

PipelineModel pipeline("pipeline_FetchDecodeExecute");
pipeline.clk(clk);

sc_start(80, SC_NS); // Step out across 8 cycles
return 0;
}

```

How the `std::optional` Stall Logic Behaves in Execution

When executing this simulation, the `MUL` instruction hits the execute stage and triggers the structural stall branch. Because the loop returns early, the `std::move` commands for lower stages are skipped, perfectly preserving the state of upstream stages without needing explicit stall logic wires:

```

--- [Cycle 10 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Bubble (Empty)
[FETCH]   Fetched New: ADD
[Pipeline Status] F: ADD | D: --- | E: ---

--- [Cycle 20 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Bubble (Empty)
[FETCH]   Passing to Decode: ADD
[FETCH]   Fetched New: MUL
[Pipeline Status] F: MUL | D: ADD | E: ---

--- [Cycle 30 ns] ---
[EXECUTE] Bubble (Empty)
[DECODE]  Processing: ADD
[FETCH]   Passing to Decode: MUL
[FETCH]   Fetched New: SUB
[Pipeline Status] F: SUB | D: MUL | E: ADD

--- [Cycle 40 ns] ---

```

```
[EXECUTE] Retired Instruction: ADD (PC: 0x1000)
[DECODE] Processing: MUL
[FETCH] Passing to Decode: SUB
[FETCH] Fetched New: AND
[Pipeline Status] F: AND | D: SUB | E: MUL

--- [Cycle 50 ns] ---
[EXECUTE] STALL: MUL busy. Remaining: 2 cycles.
[Pipeline Status] F: AND | D: SUB | E: MUL

--- [Cycle 60 ns] ---
[EXECUTE] STALL: MUL busy. Remaining: 1 cycles.
[Pipeline Status] F: AND | D: SUB | E: MUL
```

TO-DO .. option to allow ..

i) **bubbles collapsing**, or

ii) something like the **decode stage also supporting multi-cycle instructions** (i.e. the instruction struct will need to allow individual delta_cycle for each stage) whose execution should NOT be stopped/prevented by another multi-cycle instruction executing in execute stage, or even each pipe stage i (0 to 2 in the case here) taking corresponding Ci cycles based on its primary input dynamically, goal is to have a modular re-usable component for a lot of use-cases

Done (see next example): back-pressure from downstream **and** having both number of pipe stages / FIFO size programmable ..